

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“QUEUE”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data Pekan 4 ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum Struktur Data. Pada praktikum pekan keempat ini, saya mempelajari konsep **Queue** serta implementasinya dalam bahasa pemrograman Java.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu saya mengharapkan kritik dan saran yang membangun.

Saya juga mengucapkan terima kasih kepada asisten praktikum yang telah membimbing selama praktikum berlangsung.

Padang, 29 April 2026

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN	2
2.1 Pengertian Queue	2
2.2 Terminologi Dasar Queue.....	2
2.3 Operasi Dasar Queue.....	2
2.4 Implementasi dalam Java.....	3
2.4.1 QueueArray	3
2.4.2 QueueArrayDriver	4
2.4.3 QueueLinkedList	5
2.4.4 IterasiQueue	6
2.4.5 ReverseData	6
BAB III PENUTUP	8
3.1 Kesimpulan.....	8
DAFTAR PUSTAKA.....	9
LAPORAN TUGAS	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman untuk mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah **Queue**, yaitu struktur data yang menggunakan prinsip FIFO (First In First Out), dimana elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

Queue banyak digunakan dalam berbagai aplikasi seperti sistem antrian, penjadwalan proses, dan pengolahan data secara berurutan. Oleh karena itu, pemahaman tentang Queue sangat penting dalam mempelajari struktur data.

1.2 Tujuan

1. Memahami konsep dasar Queue
2. Mampu mengimplementasikan Queue dalam Java
3. Memahami prinsip FIFO dalam pengolahan data

1.3 Manfaat

1. Menambah pemahaman tentang struktur data Queue
2. Melatih kemampuan pemrograman dalam Java
3. Menjadi dasar untuk mempelajari struktur data lanjutan

BAB II

PEMBAHASAN

2.1 Pengertian Queue

Queue adalah struktur data linear yang menyimpan elemen dengan prinsip **FIFO (First In First Out)**, yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Konsep ini mirip dengan antrian di kehidupan sehari-hari, dimana orang yang datang lebih dulu akan dilayani lebih dulu.

Queue digunakan dalam berbagai aplikasi seperti sistem antrian, penjadwalan proses, dan pengolahan data secara berurutan dalam pemrograman.

2.2 Terminologi Dasar Queue

Dalam struktur data queue, terdapat beberapa istilah penting yang digunakan untuk memahami cara kerjanya.

1. **Front** merupakan posisi elemen paling depan dalam queue, yaitu elemen yang akan pertama kali diproses atau dihapus. Posisi ini juga sering disebut sebagai *head* dari queue.
2. **Rear** adalah posisi elemen paling belakang dalam queue, yaitu elemen yang terakhir ditambahkan. Posisi ini juga dikenal sebagai *tail* dari queue.
3. **Size** menunjukkan jumlah elemen yang sedang tersimpan dalam queue pada suatu waktu.
4. **Capacity** merupakan batas maksimum jumlah elemen yang dapat ditampung oleh queue.

2.3 Operasi Dasar Queue

Operasi dasar pada queue meliputi:

1. **enqueue** Digunakan untuk menambahkan elemen ke bagian belakang queue

2. **dequeue** Digunakan untuk menghapus elemen dari bagian depan queue
3. **peek / front** Digunakan untuk melihat elemen terdepan tanpa menghapusnya
4. **isEmpty** Digunakan untuk mengecek apakah queue kosong
5. **isFull** Digunakan untuk mengecek apakah queue penuh

2.4 Implementasi dalam Java

Pada praktikum pekan ke-4 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **Queue** dalam bahasa pemrograman Java.

2.4.1 QueueArray

```

1 package pekan4_2511533031;
2 public class QueueArray_2511533031 {
3     int front_3031, rear_3031, size_3031;
4     int capacity_3031;
5     int array_3031[];
6
7     public QueueArray_2511533031(int capacity) {
8         this.capacity_3031 = capacity;
9         front_3031 = this.size_3031 = 0;
10        rear_3031 = capacity - 1;
11        array_3031 = new int[this.capacity_3031];
12    }
13
14    boolean isFull_3031(QueueArray_2511533031 queue) {
15        return (queue.size_3031 == queue.capacity_3031);
16    }
17
18    boolean isEmpty_3031(QueueArray_2511533031 queue) {
19        return (queue.size_3031 == 0);
20    }
21
22    void enqueue_3031(int item) {
23        if (isFull_3031(this))
24            return;
25        this.rear_3031 = (this.rear_3031 + 1) % this.capacity_3031;
26        this.array_3031[this.rear_3031] = item;
27        this.size_3031 = this.size_3031 + 1;
28        System.out.println(item + " enqueued to queue");
29    }
30
31    int dequeue_3031() {
32        if (isEmpty_3031(this))
33            return Integer.MIN_VALUE;
34        int item = this.array_3031[this.front_3031];
35        this.front_3031 = (this.front_3031 + 1) % this.capacity_3031;
36        this.size_3031 = this.size_3031 - 1;
37        return item;
38    }
39
40    int front_3031() {
41        if (isEmpty_3031(this))
42            return Integer.MIN_VALUE;
43
44        return this.array_3031[this.front_3031];
45    }
46
47    int rear_3031() {
48        if (isEmpty_3031(this))
49            return Integer.MIN_VALUE;
50
51        return this.array_3031[this.rear_3031];
52    }
53
54    //mencetak elemen antrian
55    void display_3031() {
56        int i;
57        if (front_3031 == rear_3031) {
58            System.out.println("\nAntrian Kosong\n");
59            return;
60        }
61        //kunjungi dari belakang dan cetak
62        for (i = front_3031; i < rear_3031; i++) {
63            System.out.printf("%d <- ", array_3031[i]);
64        }
65        return;
66    }
67 }

```

Gambar 2.1

Program `QueueArray_2511533031` merupakan implementasi struktur data **Queue** menggunakan array dalam Java. Queue memiliki kapasitas tertentu dengan variabel `front_3031` sebagai posisi depan, `rear_3031` sebagai posisi belakang, dan `size_3031` sebagai jumlah elemen. Method `isFull` dan `isEmpty` digunakan untuk mengecek kondisi queue.

Operasi utama yang digunakan adalah `enqueue` untuk menambahkan elemen dan `dequeue` untuk menghapus elemen sesuai prinsip FIFO. Selain itu, terdapat method `front`, `rear`, dan `display` untuk melihat dan menampilkan isi antrian. Program ini juga menggunakan konsep circular queue agar penggunaan array lebih efisien.

2.4.2 QueueArrayDriver

```

1 package pekan4_2511533031;
2 public class QueueArrayDriver_2511533031 {
3
4     public static void main(String[] args) {
5         QueueArray_2511533031 queue_3031 = new QueueArray_2511533031(1000);
6         queue_3031.enqueue_3031(10);
7         queue_3031.enqueue_3031(20);
8         queue_3031.enqueue_3031(30);
9         queue_3031.enqueue_3031(40);
10        System.out.println("Item di depan " + queue_3031.front_3031());
11        System.out.println("Item paling belakang " + queue_3031.rear_3031());
12        System.out.println("tampilan queue");
13        queue_3031.display_3031();
14        System.out.println();
15        System.out.println(queue_3031.dequeue_3031() + " dihapus dari queue");
16        System.out.println("Item di depan: " + queue_3031.front_3031());
17        System.out.println("Item paling belakang: " + queue_3031.rear_3031());
18        System.out.println("tampilan queue setelah satu data dihapus");
19        queue_3031.display_3031();
20    }
21 }

```

Gambar 2.2

Program `QueueArrayDriver_2511533031` merupakan program utama untuk menguji implementasi queue berbasis array. Pada awalnya dibuat objek queue dengan kapasitas tertentu, kemudian beberapa elemen ditambahkan menggunakan operasi `enqueue`. Setelah itu, program menampilkan elemen terdepan dan elemen paling belakang dalam queue.

Selanjutnya, program menampilkan seluruh isi queue, kemudian melakukan operasi `dequeue` untuk menghapus elemen terdepan. Setelah penghapusan, program kembali menampilkan elemen depan, elemen belakang, serta isi queue untuk melihat perubahan yang

terjadi. Program ini menunjukkan cara kerja queue dengan prinsip FIFO.

2.4.3 QueueLinkedList

```

1 package pekan4_2511533031;
2
3 import java.util.LinkedList;
4
5
6 public class QueueLinkedList_2511533031 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_3031 = new LinkedList<>();
10        // tambah elemen (0, 1, 2, 3, 4, 5) ke antrean
11        for (int i = 0; i < 6; i++)
12            q_3031.add(i);
13        // Menampilkan isi antrean.
14        System.out.println("Elemen Antrean " + q_3031);
15        // Untuk menghapus kepala antrean
16        int hapus_3031 = q_3031.remove();
17        System.out.println("Hapus Elemen = " + hapus_3031);
18        System.out.println(q_3031);
19        // Untuk melihat antrean terdepan
20        int depan_3031 = q_3031.peek();
21        System.out.println("Kepala Antrean = " + depan_3031);
22        // Untuk melihat panjang antrean
23        int banyak_3031 = q_3031.size();
24        System.out.println("Size Antrean = " + banyak_3031);
25    }
26 }
27
28 }

```

Gambar 2.3

Program `QueueLinkedList_2511533031` merupakan implementasi struktur data **Queue** menggunakan `LinkedList` dalam Java. Pada program ini, queue dibuat menggunakan interface `Queue` dengan implementasi `LinkedList`, sehingga penyimpanan data bersifat dinamis. Beberapa elemen ditambahkan ke dalam antrean menggunakan perulangan, kemudian seluruh isi queue ditampilkan.

Selanjutnya, program melakukan operasi `remove` untuk menghapus elemen terdepan, kemudian menampilkan kembali isi antrean setelah penghapusan. Program juga menggunakan method `peek` untuk melihat elemen paling depan tanpa menghapusnya, serta `size` untuk mengetahui jumlah elemen dalam queue. Program ini menunjukkan penggunaan queue dengan prinsip FIFO menggunakan struktur `LinkedList`.

2.4.4 IterasiQueue

```

1 package pekan4_2511533031;
2 import java.util.*;
3 public class IterasiQueue_2511533031 {
4
5     public static void main(String[] args) {
6         Queue<String> q_3031 = new LinkedList<> ();
7
8         q_3031.add("Praktikum");
9         q_3031.add("Struktur");
10        q_3031.add("Data");
11        q_3031.add("dan");
12        q_3031.add("Algoritma");
13        Iterator<String> iterator = q_3031.iterator();
14        while (iterator.hasNext()) {
15            System.out.print(iterator.next() + " ");
16        }
17    }
18 }

```

Gambar 2.4

Program `IterasiQueue_2511533031` merupakan implementasi struktur data **Queue** menggunakan `LinkedList` dalam Java. Pada program ini, beberapa data bertipe `String` dimasukkan ke dalam queue menggunakan method `add`. Data yang dimasukkan membentuk sebuah kalimat yang kemudian akan ditampilkan.

Selanjutnya, program menggunakan `Iterator` untuk melakukan iterasi pada queue. Dengan menggunakan perulangan `while`, setiap elemen dalam queue ditampilkan satu per satu secara berurutan sesuai prinsip `FIFO`. Program ini menunjukkan cara mengakses seluruh elemen queue tanpa menghapus data yang ada.

2.4.5 ReverseData

```

1 package pekan4_2511533031;
2 import java.util.*;
3 public class ReverseData_2511533031 {
4
5     public static void main(String[] args) {
6         Queue<Integer> q_3031 = new LinkedList<Integer>();
7         q_3031.add(1);
8         q_3031.add(2);
9         q_3031.add(3);
10        System.out.println("sebelum reverse" + q_3031);
11        Stack<Integer> s_3031 = new Stack<Integer>();
12        while (!q_3031.isEmpty()) { // Q -> S
13            s_3031.push(q_3031.remove());
14        }
15        while (!s_3031.isEmpty()) { // S -> Q
16            q_3031.add(s_3031.pop());
17        }
18        System.out.println("sesudah reverse= " + q_3031); // {3, 2, 1}
19    }
20 }

```

Gambar 2.5

Program `ReverseData_2511533031` digunakan untuk membalik urutan data dalam **Queue** dengan bantuan struktur data **Stack**. Pada awalnya, beberapa elemen dimasukkan ke dalam queue, kemudian

ditampilkan untuk melihat urutan sebelum dilakukan proses pembalikan.

Selanjutnya, seluruh elemen dalam queue dipindahkan ke dalam stack menggunakan perulangan. Karena stack menggunakan prinsip LIFO, urutan data akan terbalik saat dimasukkan kembali ke dalam queue. Setelah itu, elemen dalam stack dikembalikan ke queue sehingga menghasilkan urutan yang telah dibalik. Program ini menunjukkan kombinasi penggunaan queue dan stack dalam pengolahan data.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa **Queue** merupakan struktur data yang menggunakan prinsip FIFO (First In First Out), dimana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Queue memiliki operasi dasar seperti enqueue, dequeue, dan peek yang digunakan untuk mengelola data secara berurutan.

Melalui praktikum ini, mahasiswa telah mampu mengimplementasikan queue menggunakan berbagai pendekatan seperti array dan LinkedList, serta menerapkannya dalam beberapa kasus seperti iterasi data dan pembalikan urutan menggunakan bantuan stack. Dengan demikian, pemahaman tentang queue dapat membantu dalam pengolahan data yang membutuhkan urutan terstruktur dalam pemrograman.

DAFTAR PUSTAKA

- [1] Programiz, "Queue Data Structure," [Online]. Available: <https://www.programiz.com/dsa/queue>. [Accessed 29 April 2026].
- [2] GeeksforGeeks, "Queue Introduction," 20 Juni 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/introduction-to-queue-data-structure-and-algorithm-tutorials/>. [Accessed 29 April 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-4.

1. Queue_2511533031

```
1 package pekan4_2511533031;
2
3 public class Queue_2511533031 {
4
5     // === DEKLARASI VARIABEL ===
6     private int front_3031;
7     private int rear_3031;
8     private int max_3031;
9     private String[] queue_3031;
10
11     // === KONSTRUKTOR ===
12     public Queue_2511533031(int max_3031) {
13         this.max_3031 = max_3031;
14         this.queue_3031 = new String[max_3031];
15         this.front_3031 = -1;
16         this.rear_3031 = -1;
17     }
18
19     // isEmpty_3031() - cek apakah antrian kosong
20     public boolean isEmpty_3031() {
21         return front_3031 == -1;
22     }
23
24     // isFull_3031() - cek apakah antrian penuh
25     public boolean isFull_3031() {
26         return rear_3031 == max_3031 - 1;
27     }
28
29     // enqueue_3031() - tambah pelanggan ke belakang antrian
30     public void enqueue_3031(String nama_3031) {
31         if (isFull_3031()) {
32             System.out.println("Antrian penuh! Tidak bisa menambah data.");
33             return;
34         }
35         if (isEmpty_3031()) {
36             front_3031 = 0;
37         }
38         rear_3031++;
39         queue_3031[rear_3031] = nama_3031;
40         System.out.println("Data berhasil ditambahkan ke antrian");
41     }
42
43     // dequeue_3031() - hapus pelanggan dari depan antrian
44     public void dequeue_3031() {
45         if (isEmpty_3031()) {
46             System.out.println("Antrian kosong! Tidak ada data yang dihapus.");
47             return;
48         }
49         String dilayani_3031 = queue_3031[front_3031];
50
51         // Geser semua elemen ke depan
52         for (int i_3031 = front_3031; i_3031 < rear_3031; i_3031++) {
53             queue_3031[i_3031] = queue_3031[i_3031 + 1];
54         }
55
56         queue_3031[rear_3031] = null;
57         rear_3031--;
58
59         if (rear_3031 < front_3031) {
60             front_3031 = -1;
61             rear_3031 = -1;
62         }
63
64         System.out.println(dilayani_3031 + " telah dilayani");
65     }
66
67     // display_3031() - tampilkan seluruh isi antrian
68     public void display_3031() {
69         if (isEmpty_3031()) {
70             System.out.println("Antrian kosong!");
71             return;
72         }
73         System.out.println("Isi antrian:");
74         for (int i_3031 = front_3031; i_3031 <= rear_3031; i_3031++) {
75             System.out.println((i_3031 - front_3031 + 1) + ". " + queue_3031[i_3031]);
76         }
77     }
78
79     // reverse_3031() - balik seluruh isi antrian
80     public void reverse_3031() {
81         if (isEmpty_3031()) {
82             System.out.println("Antrian kosong!");
83             return;
84         }
85     }
```

```

85
86     int kiri_3031 = front_3031;
87     int kanan_3031 = rear_3031;
88
89     while (kiri_3031 < kanan_3031) {
90         String temp_3031 = queue_3031[kiri_3031];
91         queue_3031[kiri_3031] = queue_3031[kanan_3031];
92         queue_3031[kanan_3031] = temp_3031;
93
94         kiri_3031++;
95         kanan_3031--;
96     }
97
98     System.out.println("Isi antrian setelah dibalik:");
99     for (int i_3031 = front_3031; i_3031 <= rear_3031; i_3031++) {
100         System.out.println((i_3031 - front_3031 + 1) + ". " + queue_3031[i_3031]);
101     }
102 }
103 }

```

Gambar 3.1

Pada program ini dibuat sebuah kelas **Queue_2511533031** yang digunakan untuk merepresentasikan struktur data antrian (queue) berbasis array. Variabel **front_3031** dan **rear_3031** digunakan untuk menandai posisi depan dan belakang antrian, sedangkan **max_3031** menyimpan kapasitas maksimum antrian, dan **queue_3031** merupakan array yang menyimpan data berupa nama pelanggan. Konstruktor digunakan untuk menginisialisasi ukuran antrian serta mengatur kondisi awal antrian dalam keadaan kosong dengan nilai **front_3031** dan **rear_3031** sama dengan -1. Selain itu, terdapat method **isEmpty_3031()** dan **isFull_3031()** yang berfungsi untuk mengecek apakah antrian kosong atau sudah penuh.

Operasi utama pada antrian diimplementasikan melalui beberapa method, yaitu **enqueue_3031()** untuk menambahkan data ke bagian belakang antrian, **dequeue_3031()** untuk menghapus data dari bagian depan antrian dengan mekanisme pergeseran elemen, serta **display_3031()** untuk menampilkan seluruh isi antrian. Selain itu, terdapat method **reverse_3031()** yang berfungsi untuk membalik urutan antrian dengan cara menukar elemen dari posisi depan dan belakang hingga bertemu di tengah. Dengan adanya method-method tersebut, program ini dapat mensimulasikan proses antrian secara lengkap, mulai dari penambahan, penghapusan, penampilan, hingga pembalikan data dalam antrian.

2. AntrianLoket_2511533031

```
1 package pekan4_2511533031;
2
3 import java.util.*;
4 public class AntrianLoket_2511533031 {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         Queue_2511533031 antrian = new Queue_2511533031(5);
9         int pilihan_3031;
10
11         do {
12             System.out.println("=== PROGRAM ANTRIAN LOKET ===");
13             System.out.println("1. Tambah Antrian");
14             System.out.println("2. Hapus Antrian");
15             System.out.println("3. Tampilkan Antrian");
16             System.out.println("4. Reverse");
17             System.out.println("5. Keluar");
18             System.out.print("Pilih menu: ");
19             pilihan_3031 = sc.nextInt();
20             sc.nextLine(); // buang newline
21
22             switch (pilihan_3031) {
23                 case 1:
24                     System.out.print("Masukkan nama pelanggan: ");
25                     String nama_3031 = sc.nextLine();
26                     antrian.enqueue_3031(nama_3031);
27                     break;
28
29                 case 2:
30                     antrian.dequeue_3031();
31                     break;
32
33                 case 3:
34                     antrian.display_3031();
35                     break;
36
37                 case 4:
38                     antrian.reverse_3031();
39                     break;
40
41                 case 5:
42                     System.out.println("Program selesai");
43                     break;
44
45                 default:
46                     System.out.println("Pilihan tidak valid!");
47             }
48
49             while (pilihan_3031 != 5);
50
51             sc.close();
52         }
53     }
54 }
```

Gambar 3.2

Pada program ini dibuat kelas `AntrianLoket_2511533031` yang berfungsi sebagai driver atau pengendali utama dalam menjalankan program antrian. Di dalam method `main`, digunakan objek dari kelas `Queue_2511533031` untuk mengelola data antrian dengan kapasitas tertentu. Selain itu, digunakan `Scanner` untuk menerima input dari pengguna, seperti pilihan menu dan data nama pelanggan. Variabel `pilihan_3031` digunakan untuk menyimpan pilihan menu yang akan dijalankan oleh pengguna.

Program dijalankan menggunakan perulangan `do-while` sehingga menu akan terus ditampilkan sampai pengguna memilih opsi keluar. Struktur `switch-case` digunakan untuk mengatur jalannya program berdasarkan pilihan pengguna, seperti menambahkan data ke antrian menggunakan `enqueue_3031()`, menghapus data menggunakan

`dequeue_3031()`, menampilkan isi antrian melalui `display_3031()`, serta membalik urutan antrian dengan `reverse_3031()`. Dengan adanya driver ini, pengguna dapat berinteraksi langsung dengan sistem antrian sehingga seluruh operasi pada struktur data queue dapat dijalankan secara dinamis melalui menu yang tersedia.